

Assignment 3 — TODO Implementation Log

Intelligent E-commerce Supply Chain Manager · UCLA Extension Agentic AI, Week 3

Covers all 12 graded TODOs across the five workflow components. For each: what was implemented, why, and how it was verified.

Contents

Master Planner Agent — MP-1, MP-2	18 pts
Demand Forecasting Engine — DF-1, DF-2, DF-3	18 pts
Inventory Optimization Planner — EOQ-1, EOQ-2, EOQ-3	22 pts
Supplier Performance Monitor — SP-1	14 pts
Logistics Coordinator — LG-1, LG-2, LG-3	18 pts
Final Synthesis — FN-1	10 pts
Appendix: infrastructure fixes required to make the starter kit run	

Master Planner Agent — 18 pts

HTTP Request node calling Claude Sonnet · decomposes the business goal into ordered subgoals, chooses `next_subgoal`

MP-1 Output JSON schema LO-1 · easy · 8 pts

Defined the required output contract for the planner: a `plan_id`, a short `reasoning` string, an ordered `subgoals` array (each with `id` / `type` / `rationale`), and a `next_subgoal` field constrained to exactly one of the four literal strings `"forecast"` | `"inventory"` | `"supplier"` | `"logistics"`. The prompt explicitly forbids markdown fences or preamble, since the downstream *Parse Plan Code* node does a bare `JSON.parse` on the entire response.

Why: the Switch node routes purely on string-matching `next_subgoal`. If the model emits anything else — prose, a differently-named field, a typo — the plan silently falls into the Switch's unhandled "extra" branch and nothing happens. The schema is the entire contract holding the rest of the graph together; getting it precise (exact literals, raw JSON only) was the whole task.

✓ **Verified** — confirmed the planner reliably returns valid, schema-conformant JSON and the Switch correctly routes on `next_subgoal` across every branch (forecast/inventory/supplier/logistics), tested via real end-to-end runs.

MP-2 Worked HTN-decomposition examples LO-1 · medium · 10 pts

Added two worked examples to the system prompt:

- **Example A** — the assignment's actual Q1 goal, decomposing into all four subgoals (`forecast` → `inventory` → `supplier` → `logistics`), with the rationale spelled out: inventory decisions need a demand forecast first, logistics needs to know which SKUs are urgent, and supplier scoring has no dependency so it's free to slot in anywhere.
- **Example B** — a supplier-only goal ("audit supplier reliability...") that decomposes into a *single* subgoal. This one exists specifically to demonstrate that the planner should *not* manufacture forecast/inventory/logistics subgoals just because the four-way vocabulary exists — only decompose into what the goal actually requires.

Closed with an explicit general rule: `next_subgoal` is always the first subgoal in the plan's own ordering whose dependencies are already satisfied — never a later one picked out of urgency.

Why: per the planning primer, single-call decomposition is prone to inconsistent vocabularies and drift. Examples anchor the model on two failure modes at once — wrong *order* (optimizing inventory before you have a forecast) and wrong *scope* (padding out subgoals that don't belong). Without a scope example, a goal like "just check my suppliers" risks getting a full four-subgoal plan the goal never asked for.

✓ **Verified** — re-ran the workflow after adding the examples — the planner produced exactly the ordering Example A describes (`forecast`, `inventory`, `supplier`, `logistics` , `next_subgoal`: "forecast"), matching the worked example almost verbatim.

Demand Forecasting Engine — 18 pts

Code node (classical) → HTTP Request node (LLM revision) · trailing-average forecast, revised for context the math can't see

DF-1

`movingAverage()`

LO-3 · easy · 6 pts

```
function movingAverage(series) {
  if (series.length === 0) return 0;
  const last3 = series.slice(-3);
  const sum = last3.reduce((acc, r) => acc + Number(r.units_sold), 0);
  return sum / last3.length;
}
```

Why: the simplest possible trend estimator — mean of the last (up to) three months. This is a deliberately naive, STRIPS-style assumption that recent history predicts the near future. It's supposed to be wrong for the two planted scenarios (a viral SKU, a declining SKU); catching that is the seasonal index's and the LLM step's job, not this function's.

✓ **Verified** — SKU-001's last three months (Oct/Nov/Dec = 46, 28, 23) average to 32.33, matching a hand calculation. Empty-series and single-item edge cases return 0 and the single value respectively, as specified.

DF-2

seasonalIndex()

LO-3 · medium · 8 pts

```
function seasonalIndex(series, targetMonth) {
  if (series.length === 0) return 1.0;
  const monthPart = targetMonth.split("-")[1];
  const monthRows = series.filter(r => r.month.split("-")[1] === monthPart);
  if (monthRows.length === 0) return 1.0;
  const mean = rows => rows.reduce((acc, r) => acc + Number(r.units_sold), 0) / rows.length;
  const monthMean = mean(monthRows);
  const overallMean = mean(series);
  if (overallMean === 0) return 1.0;
  return monthMean / overallMean;
}
```

Why: separates trend from seasonality — a 1.5 multiplier means "this calendar month historically runs 50% above average." Multiplying `movingAverage` × `seasonalIndex` gives a forecast that tracks both recent momentum and calendar patterns. Guards against empty series, no matching month, and zero overall mean all default to 1.0 (a neutral multiplier) rather than crashing or returning garbage.

✓ **Verified** — tested against SKU-007 (Wool Gloves) — whose data happens to show the assignment's planted viral spike (Sept–Dec climbing from 100 to 518 units/month). Produced sensible index values (0.94 for a January target) without needing any SKU-specific logic.

DF-3

Forecast Context Adjuster prompt

LO-5 · easy · 4 pts

The prompt defines which contextual signals justify overriding the classical number (viral mentions, promotions, weather anomalies, supply disruptions, competitor stockouts), and bounds how far the LLM can deviate without strong justification: under 10% needs only a one-line reason, 10–30% needs a named signal, over 30% requires an explicit high-confidence signal spelled out in the reasoning field. Absent a qualifying signal, the instruction is to return the classical forecast unchanged.

Why: this is Pattern A from the planning primer — classical baseline, LLM revision. The bound exists so a 50% revision needs justification a 5% revision doesn't; without it, the LLM could silently replace a reproducible number with an unaccountable one.

✓ **Verified** — ran across all 15 SKUs — the LLM correctly returned `delta_pct: 0` for every one, including SKU-007. This is *not* a bug: the `notes` field in `demand-forecast.js` is always an empty array (no external context ever gets attached in this dataset), so there was genuinely no signal to act on, and the model correctly said so rather than inventing one. Catching SKU-007's viral spike turned out to be the EOQ branch's job (see EOQ-2 below) — a good illustration of the primer's point that different branches catch different classes of classical-model failure.

Inventory Optimization Planner — 22 pts

Code node (classical EOQ) → LLM exception handler for flagged SKUs only

EOQ-1

eoq() formula

LO-2 · medium · 8 pts

```
function eoq(D, S, H) {
  if (D === 0 || H === 0) return 0;
  return Math.round(Math.sqrt((2 * D * S) / H));
}
```

Why: Wilson's EOQ (1934) — the canonical closed-form OR result. One line of math, correct exactly when its assumptions (constant demand, no stockouts, stable costs) hold, which is precisely why EOQ-2 exists.

✓ **Verified** — SKU-001: D=729, S=\$50, H=\$1.20 → Q*=246, matching $\text{sqrt}(2 \cdot 729 \cdot 50 / 1.2) \approx 246.5$ by hand.

EOQ-2

detectViolations()

LO-4 · hard · 10 pts

Four independent checks, each returning a flag string if it fires:

Flag	Condition
viral_spike	mean(last 3 months) > 2.5 × mean(prior 9 months)
declining	mean(last 3 months) < 0.5 × mean(first 3 months)
low_velocity	annual demand < 60 units
long_lead_time	lead time > 28 days <i>and</i> demand is volatile (coefficient of variation > 0.4)

Why: per the primer's STRIPS-assumption framing, this is where the workflow admits its own formula might be wrong and says so explicitly, rather than confidently returning a bad number. A SKU can trip multiple flags — that's expected and gets surfaced to the LLM exception handler as extra context.

✓ **Verified** — ran against all 15 real SKUs — correctly caught both scenarios the assignment brief plants in the data: **SKU-007** (Wool Gloves, only 18 units on hand vs. an 80-unit reorder point) flags `viral_spike`; **SKU-013** (Wired Earbuds, 540 on hand vs. 80 reorder point — exactly the "6.8×" the brief describes) flags `declining`. It also generalized to flag three winter SKUs (long lead time + volatile demand) with no SKU-specific hardcoding anywhere in the function.

EOQ-3 Inventory Exception Handler prompt LO-4 · easy · 4 pts

Per-flag guidance: `viral_spike` → reorder above the EOQ figure, sized off the recent trend rather than the (already-known-lagging) formula; `declining` → markdown or liquidate, qty 0; `low_velocity` → a small fixed reorder approximating 30 days of demand rather than the numerically unstable EOQ; `long_lead_time` → reorder at the EOQ figure but flag the timing risk for a human. Multiple flags are resolved by addressing the most urgent one first (stockout risk beats a stability nuisance).

Why: Pattern B from the primer — classical solver, LLM exception handler. This is the prompt that decides *what to do* once the classical layer has already said "don't trust my number here."

✓ **Verified** — SKU-007 → reorder , qty 550 (above the EOQ figure of 431, explicitly sized off the accelerating trend). SKU-013 → liquidate , qty 0, citing the 6.75× reorder-point ratio. Both align exactly with the guidance above.

⚠ **Scaffold bug found** — the starter wiring sent *every* SKU to this LLM node regardless of whether it was flagged — contradicting the primer's own description of Pattern B ("no LLM call" for clean SKUs) and blowing the stated cost budget roughly 3×. Fixed by adding an IF node (`Inventory Needs LLM Review?`) that gates on `needs_llm_review` , routing clean SKUs straight to the merge point with their EOQ numbers intact. This is a topology change (one new node) — flagged and approved before implementing, per the rubric's allowance for justified departures. Confirmed via a live test: LLM calls dropped from 15 to exactly 5, matching the number of actually-flagged SKUs.

Supplier Performance Monitor — 14 pts

Single HTTP Request node — the whole component is one LLM call scoring all six suppliers at once

SP-1 4-dimension weighted rubric LO-5 · medium · 14 pts

Dimension	Weight	Rationale
Reliability (on_time_rate)	35%	Weighted highest — a late shipment cascades directly into the stockouts this whole workflow exists to catch.
Quality (1 - defect_rate)	30%	Nearly as high — defects compound into returns and lost trust, and unlike lead time, can't be buffered with safety stock.
Lead time (avg_lead_time_days)	20%	Lower weight — EOQ and reorder points already partially absorb lead-time length; it's a planning input more than a failure mode.
Cost (payment_terms_days proxy)	15%	Lowest — affects margin, not whether the order arrives correct and on time.

Each dimension is min-max normalized to 0–100 *across the six suppliers in the roster* (not against an absolute external benchmark) before the weights are applied, so the score is always a relative read of this

specific supplier pool. Tiers: `preferred` ≥ 80 , `approved` 60–79, `watch` 40–59, `deprioritize` < 40 .

Why: per the primer, this is where an LLM genuinely beats a classical MCDM framework like AHP — AHP needs a domain expert to hand-elicited pairwise weights; an explicit, defensible weighting can be specified directly in the prompt and applied consistently.

✓ **Verified** — spot-checked the model's arithmetic against its own normalization: e.g. SUP-002 (worst on-time rate and defect rate in the roster) correctly scored 0 on both the reliability and quality dimensions since it's the roster minimum on each. Weighted totals and tier assignments were all internally consistent with the stated formula. No supplier crossed 80 (max was ~85 in one run) — a fair reflection of the data, not a scoring bug.

△ **Scaffold bug found** — the model initially showed its normalization work (a full markdown table plus arithmetic) before the JSON, violating the "raw JSON only" instruction and burning roughly 2–3× the intended tokens. Fixed by adding an explicit "reason silently — the first character of your response must be [" instruction. Confirmed the fix: subsequent responses started directly with [, no preamble.

Logistics Coordinator — 18 pts

HTTP Request node (LLM, soft constraints) → IF node → Code node (classical greedy fallback)

LG-1 **Hard + soft constraint reasoning** LO-5 · easy · 6 pts

The prompt first has the model filter the carrier catalogue against five hard constraints (origin/destination region match, transit time within deadline, weight capacity, perishability support) — the same five `pickCheapestFeasible()` checks classically, so the two branches have comparable scope. Among qualifying options, cost is the primary soft preference, but when two or more land within roughly 15% of each other, the tie is broken by preferring more time slack (transit days well under the deadline) rather than pure lowest cost.

Why: Pattern C from the primer — LLM as router over soft constraints a formula can't cleanly encode (which carrier's slightly-higher-cost option is worth it for the extra buffer).

LG-3 `use_classical_fallback` **logic** LO-4 · easy · 2 pts

Folded into the same prompt paragraph as LG-1: `use_classical_fallback: true` only when one option is unambiguously cheapest among the qualifying set (no real judgment call); `false` only when the LLM actually had to exercise judgment — a genuine cost tie broken by time slack, or a case where nothing satisfies every hard constraint and the model is recommending a least-bad compromise.

Why: this rule means that whenever fallback fires, the LLM's own answer and the deterministic planner's answer should *agree* — a built-in audit check. Deferring is a legitimate LLM decision, not an evasion of judgment.

✓ **Verified** — across the three synthetic requests: REQ-001 (a genuine 15%-band cost tie between two air carriers) → LLM broke the tie by time slack, both branches independently arrived at the same carrier (SHP-004, \$10,030). REQ-002 (no carrier meets the 3-day deadline) → correctly returned `chosen_option_id: null` with `use_classical_fallback: false`, since this genuinely required judgment about the least-bad compromise. REQ-003 (single qualifying option) → unambiguous, `use_classical_fallback: true`, and the classical planner independently confirmed the same choice (SHP-008, \$1,980).

LG-2 `pickCheapestFeasible()`

LO-2 · hard · 10 pts

```
function pickCheapestFeasible(req, options) {
  const feasible = options.filter(o =>
    o.origin_region === req.origin_region &&
    o.destination_region === req.dest_region &&
    Number(o.transit_days) <= Number(req.deadline_days) &&
    Number(o.max_weight_kg) >= Number(req.weight_kg) &&
    (!req.perishable || o.supports_perishable === true || o.supports_perishable === "true")
  );
  if (feasible.length === 0) return null;
  const priced = feasible.map(o => ({
    ...o, total_cost_usd: Number(req.weight_kg) * Number(o.cost_per_kg),
  }));
  return priced.reduce((cheapest, o) =>
    o.total_cost_usd < cheapest.total_cost_usd ? o : cheapest);
}
```

Why: the deliberately simple classical baseline the LLM branch gets compared against. Filter, then min-by-cost — greedy rather than exhaustive, on purpose, so the two branches have comparable search scope and the comparison between them is meaningful.

✓ **Verified** — tested against all three requests by hand before wiring into n8n: REQ-001 correctly picks SHP-004 over the initially-plausible SHP-003 (cheaper per-kg once weight is factored in); REQ-002 correctly returns `null` (no North America ground carrier meets a 3-day deadline); REQ-003 correctly picks the ocean carrier SHP-008 over the far more expensive air option SHP-007. All three matched the live n8n execution exactly.

⚠ **Scaffold bug found** — **Three separate bugs** had to be fixed before this branch could run correctly at all:

1. **13 LLM calls instead of 3** — `Build Shipping Requests` was returning both the 3 real requests *and* the 10 carrier options as one mixed list, and since the LLM node runs once per input item, it fired once per option too (with the model confusedly trying to treat a carrier option as if it were a shipping request). Fixed by having that node return only the 3 requests — the options were already available downstream via a cross-reference, so nothing else needed the mixed list.
2. **The classical-fallback branch was dead code** — the IF node checked `$json.use_classical_fallback` directly on the LLM node's output, but that node returns the raw, unparsed Anthropic response envelope (the field is buried inside `content[0].text` as a JSON string).

The condition could never evaluate true. Added a `Parse Logistics Decision Code` node (mirroring the existing `Parse Plan` pattern) so the field is actually readable.

3. **A bug in my own fix to bug #2** — the new parse node used `$json` (singular) instead of looping over `$input.all()`, so it silently processed only the first of three requests. Fixed to loop properly.

All three confirmed fixed via live re-runs; see the appendix for the one remaining known limitation (a Final-Output aggregation edge case, not a correctness bug).

Final Synthesis — 10 pts

Code node — the last node in the graph, downstream of the merge point for whichever branch fired

FN-1 Aggregate into a business-readable summary LO-1 · medium · 10 pts

Produces `{ plan_id, executed_subgoal, key_findings, next_recommended_subgoal }`. An `extractLLMJson` helper detects and unwraps raw Anthropic response envelopes wherever they appear (some branches parse earlier in the graph, some don't), and flattens the Supplier Monitor's single array-shaped response so `key_findings` is always a flat list regardless of which branch produced it.

`next_recommended_subgoal` is derived by looking up, in the Master Planner's *own* `subgoals` array, whichever subgoal comes immediately after the one that just executed — not a hardcoded sequence.

Why: per the primer, this field is "the seed of a real planning loop" — feeding it back into the planner on a subsequent run would advance the plan one step at a time (not wired up in this assignment, but the mechanism is real). Deriving it from the planner's own ordering rather than a fixed array keeps it faithful to genuine dynamic HTN decomposition — if the planner ever produces a different subgoal order for a different goal, this still does the right thing.

✓ **Verified** — on a clean forecast-branch run: `executed_subgoal: "forecast"`, `next_recommended_subgoal: "inventory"` — correctly read off the plan's own stated ordering.

Appendix: infrastructure fixes required to make the starter kit run

None of these were TODOs — they were pre-existing bugs in the provided scaffold/environment that blocked all 12 TODOs from ever being testable. Documented here for completeness and because they're relevant material for `analysis.md`.

- **n8n version pin.** `docker-compose.yml` was pinned to `n8n:latest`, which now resolves to a 2.x release with a broken/gated production-webhook path (unrelated to this assignment's workflow, which uses a Manual Trigger — found while testing a different exercise in this repo, but the pin affects this container too).
- **Fan-in scheduling bug.** The four parallel CSV-read branches feeding `Build Context Summary` relied on a "wait for all sources" behavior n8n's default scheduler doesn't actually provide for plain multi-wire convergence. Fixed by setting `executionOrder: "v0"` in the workflow settings (a one-line, non-invasive fix).

- **Wrong data source in three Code/LLM nodes.** Demand Forecast Engine , Inventory EOQ Planner , and Supplier Performance Monitor all read `$input.all()` for their CSV data, but their only wired input is the routed *plan* object from the Switch node. Fixed to use `$('Read ... JSON').all()` cross-references, matching the pattern two sibling nodes in the same workflow already used correctly.

One remaining known limitation (not a correctness bug): when a logistics-branch run has a genuine mix of fallback and non-fallback decisions across the three requests, Final Output produces two separate, individually-correct partial summaries instead of one merged one. This traces to an n8n execution-engine quirk when two different completion paths are simultaneously active within a single run feeding one Merge node. An attempted fix (splitting the Merge node's two paths onto separate input ports) was tried and reverted after it caused a worse problem — stale data from unrelated earlier test executions bleeding into results. Individual finding data is confirmed accurate in every case; only the single-summary aggregation is incomplete in this specific edge case.